API Integration Assistant

Proposed ingestion and retrieval design

Use LangChain to turn a customer's business request into an integration guide: the applicable workflow, ordered API calls, relevant fields, value sources, request examples and source references. The first prototype provides guidance for fictional APIs; live API execution is outside its proposed scope.



Ingestion pipeline

On initial load and source changes, parse API specifications, resolve schema references, and preserve request parameters and request/response schemas. Normalize supplied workflows and business rules into reviewed records with applicability conditions, ordered steps, branches, field rules and output-to-input bindings. Flag missing or conflicting definitions.

Keep original sources and complete records with versions and source locations. Embed workflow descriptions and API/rule passages into a local vector store, with record IDs, document type and version metadata. Refresh affected records and embeddings on updates. The prototype uses JSON/YAML for canonical records plus a local vector index.

Prompt lookup and answer generation

1. Retrieve and select. Embed the prompt with relevant conversation context. Search workflow descriptions in the vector store, filtering by known conditions and current versions. The LLM checks candidate applicability and returns a known ID, clarification, or no match. Validate its selection; similarity alone does not establish a supported match.

2. Fetch complete evidence. Read the selected workflow by ID and version. Fetch every referenced operation by API identity, version and operationId, including resolved schemas, linked rules and examples. Apply documented branches and data bindings. Scope any supplementary passage search to the selected workflow and API versions.

3. Generate and check. Combine API-required fields with scenario rules. Explain values from user input, allowed enums, configuration and earlier responses. Generate ordered calls and synthetic payloads with citations. Check required fields, schema constraints and encoded workflow rules; report unsupported or conflicting evidence. Schema checks do not prove successful execution.

Prototype and acceptance

Use personal and business account-opening workflows over three fictional operations: createCustomer → verifyIdentity → createAccount. Pass customerId forward; require successful verification. Distinguish company-only fields while preserving schema-required inputs. Expose retrieved candidate IDs to demonstrate vector lookup.

Through a minimal chat UI, demonstrate a supported request with the correct sequence, field/value explanations and valid examples; clarification for an incomplete request; and an explicit no-match result for an unsupported request. Expose source versions and output-to-input mappings in the answer.

Implementation and decisions to confirm

Use a fixed LangChain retrieval and generation chain. Search and fetch functions can also be exposed as tools; add MCP when multiple clients need access. The registry supplies ordering; the vector index supplies candidates.

Confirm the first scenario and authoritative specs/rules; who reviews workflow order and exceptions; approved chat/embedding models, storage and UI; and whether executable mock requests are required.